

Reviewer Pack — Minimal Noncommutative Entropy of Matrix Algebras vs BP_Read...

Entry 41ac56b6f189 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 18:28:21 UTC

Minimal Noncommutative Entropy of Matrix Algebras vs BP_ReadTwice Circuit Size

Entry ID: 41ac56b6f189 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 18:28:21 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

Statement:

[‘For all read-twice Boolean functions f , the minimal noncommutative entropy of the algebra generated by the support of any matrix representation of f is upper bounded by $\log(n)$ where n is the size of the function.’, ‘The minimal noncommutative entropy for the trivial BP (where each input bit decides independently) is lower bounded by $\Omega(n^2)$.’, ‘For all distinct matrix representations of f , their noncommutative entropy sum is at least as large as that of any single representation.’]

Rationale (proposer’s reasoning):

[‘Noncommutative algebra may provide a richer framework to analyze the complexity of BP_readtwice circuits, potentially revealing structural properties not visible in classical linear algebra.’, ‘The minimal noncommutative entropy measures the ‘non-commutativity’ or ‘complexity’ of an algebraic structure and could serve as a useful invariant for circuit complexity.’, ‘This conjecture builds on the idea of using entropic invariants to distinguish between different types of circuits, similar to how tropical geometry has been used in analyzing Boolean functions.’]

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8a8882e68ccf6219

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ read-twice Boolean functions, the sum of noncommutative entropy values across all matrix representations exceeds $\log(n)$ and is at least $\Omega(n^2)$, with a confidence interval below 5%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def calculate_noncommutative_entropy(matrix):
        n = len(matrix)
        entropy = 0
        for i in range(n):
            for j in range(i+1, n):
                if matrix[i][j] != matrix[j][i]:
                    entropy += math.log2(2)
        return entropy

    def generate_matrix_representation(function):
        n = int(math.log2(len(function)))
        matrix = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if function[i*2**j] != function[j*2**i]:
                    matrix[i][j] = 1
                    matrix[j][i] = 1
        return matrix

    def sum_noncommutative_entropy(functions):
        total_entropy = 0
        for function in functions:
            matrix = generate_matrix_representation(function)
            entropy = calculate_noncommutative_entropy(matrix)
```

```

        total_entropy += entropy
    return total_entropy

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    function = generate_read_twice_function(n)
    functions = [function] * 30
    total_entropy = sum_noncommutative_entropy(functions)

    if total_entropy <= math.log2(n):
        return {
            "metric_name": "Noncommutative Entropy Sum",
            "metric_value": total_entropy,
            "instances_tested": len(functions),
            "conjecture_holds": False,
            "counterexample": f"n={n}, Total Entropy <= log(n)"
        }

    if total_entropy < n**2:
        return {
            "metric_name": "Noncommutative Entropy Sum",
            "metric_value": total_entropy,
            "instances_tested": len(functions),
            "conjecture_holds": False,
            "counterexample": f"n={n}, Total Entropy < n^2"
        }

    results.append(total_entropy)

mean_entropy = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean_entropy)**2 for x in results) / len(results))

return {
    "metric_name": "Noncommutative Entropy Sum",
    "metric_value": mean_entropy,
    "instances_tested": len(results),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_entropy = sum(x["metric_value"] for x in results) / len(results)
    std_dev = math.sqrt(sum((x["metric_value"] - mean_entropy)**2 for x in results) / len(results))
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_entropy} std={std_dev} support_fraction={support_fraction}")
elif any(not x["conjecture_holds"] for x in results):
    first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\n={results[0]['metric_value']}\n" first_failing_seed)
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b47606e2.py", line 96, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b47606e2.py", line 57, in run_trial
    total_entropy = sum_noncommutative_entropy(functions)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b47606e2.py", line 46, in sum_noncommutative_entropy
    matrix = generate_matrix_representation(function)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b47606e2.py", line 38, in generate_matrix_representation
    if function[i*2**j] != function[j*2**i]:
       ~~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14999
2	preregistration	ollama_remote	glm4:latest	0	0	9728
3	novelty	ollama_remote	glm4:latest	0	0	8554

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8919
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44331
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14825
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9642
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11821
9	judge	ollama_remote	glm4:latest	0	0	39587

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162405 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/41ac56b6f189.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/41ac56b6f189.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/41ac56b6f189.tar.gz* (if generated)